VizFix: A Bug-Report-Grounded Oracle for Verifying Automated UI Repairs

Introduction

Automated program repair (APR) agents are increasingly used to patch bugs in Android apps, yet deciding whether a patch actually fixes a UI bug remains unsolved. UI bugs express correctness through visual and behavioral properties (a misaligned component, a stale value after screen rotation, a button that responds but performs the wrong action), so no simple pass/fail check exists. Current practice hands that judgment back to the repair agent, and agents treat almost any visible screen change as evidence of a fix. I call this the *oracle problem for UI repair*, and it is a central bottleneck for trustworthy agentic software engineering. This project will measure how often agent self-verification fails, build an external oracle that grounds verification in the bug report itself, and show that this oracle improves which patches repair pipelines accept.

Background

APR research has traditionally judged patches with test suites or by re-running a crash. Android UI bugs rarely come with tests, and our group's recent study of AI repair across 50 bugs in 50 open-source Android apps [2] showed that even re-execution is not enough: only 25 of the 50 AI-generated repairs were genuinely correct. Ten resolved the surface symptom without fixing the intended fault, and ten made the defect *appear* resolved while it remained latent. I contributed to this study by writing its structured bug reports, helping develop its repair-quality taxonomy, and validating its emulator reproduction scripts; a binary "does it still fail?" check would have accepted many of these patches. A better oracle needs to know *which* part of the screen the bug report is about and *how* that part should change. FRAME [1] provides the first ingredient: it fuses an app's UI structure (the view hierarchy) with visual semantics into component-level embeddings, making it possible to locate a specific component across two versions of a screen. No prior work, however, systematically measures how accurately agents verify their own UI fixes, or combines bug-report semantics with component localization to verify them.

Research Goal 1: Measure the Oracle Problem (Year 1)

H1 (Overconfidence): When shown buggy and patched screens, current vision-language model (VLM) agents will frequently accept cosmetic patches as genuine fixes while reporting high confidence. I will construct **BugFix-Oracle**, a benchmark of at least 200 Android UI bugs across 50 apps drawn from open-source issue trackers (AndroR2, Themis). Each bug is paired with its genuine fix and two to four cosmetic patches: screen-visible changes, generated with operators adapted from the MDroid+ mutation framework, that do not deliver the behavior the bug report specifies. I will evaluate four current VLMs (proprietary and open-weight) under three input conditions: screenshots only; screenshots plus the bug report; and screenshots, bug report, and code diff. For every verdict I record the fix/no-fix judgment, stated confidence, and error type. Primary metrics are F1 score on genuine-fix detection and Expected Calibration Error (ECE), which quantifies the gap between confidence and accuracy. I expect F1 below 0.55 alongside mean confidence above 0.80, which would show that agent self-verification cannot be trusted as currently practiced.

Research Goal 2: Design a Grounded Oracle (Year 2)

H2 (Grounded oracle): An oracle that compares the specific component named in the bug report, in the direction the report specifies, will significantly outperform VLM self-verification at separating genuine fixes from cosmetic patches. The oracle works in three stages. (1) *Report parsing:* a structured LLM prompt extracts the affected component, expected behavior, reproduction path, and observable failure symptom. (2) *Component localization:* FRAME embeddings match that component description against the view hierarchies of the pre- and post-patch screens, returning ranked candidate components. (3) *State comparison:* for each localized component, the oracle compares its FRAME embedding, bounding box, and accessibility attributes before and after the patch. A component counts as fixed only when the change matches the report's expected behavior, not merely when something changed. For example, if a report says a cart total shows a stale price after rotation, a patch that bolds the label changes the right component in the wrong way and is rejected. I will evaluate localization accuracy (IoU against manually annotated component bounding boxes) and fix-classification F1 on BugFix-Oracle, compared against the Goal 1 baselines.

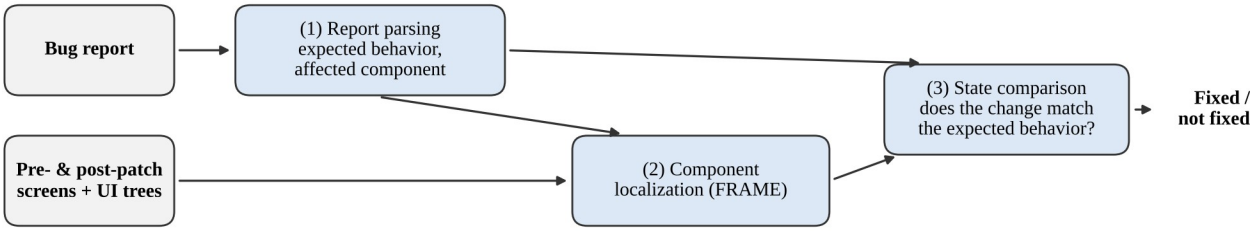


Figure 1. The grounded oracle: the bug report defines what should change; FRAME finds where; the comparison checks whether it did.

Research Goal 3: Integrate and Evaluate in Repair Pipelines (Year 3)

I will replace the self-verification step in two open-source APR agents with the grounded oracle. On a held-out set of 100 bugs not used during oracle development, I will measure correct-patch selection rate, false-positive rate (cosmetic patches accepted), and runtime overhead across four conditions: agent self-verification, pixel-level screen diff, FRAME similarity without report grounding, and the full oracle. A final study will apply the oracle to 25 real agent-authored pull requests from open-source Android repositories, with manually adjudicated ground truth, to measure how often it correctly blocks a cosmetic patch from being merged.

Risks and Mitigation

Three risks could limit the oracle. First, LLM report parsing may misread vague bug reports; I will validate extracted fields against a manually annotated subset, and the structured reports and reproduction scripts our group built for [2] provide a high-quality starting set. Second, FRAME may struggle to localize custom-drawn components that lack clear view-hierarchy entries; for these, the oracle will fall back to accessibility attributes and on-screen text, and I will report localization failures separately so they do not hide inside overall accuracy. Third, mutation-generated cosmetic patches may not reflect how real agents fail, so the benchmark will also include the misleading agent repairs observed in [2].

Intellectual Merit

This project makes four contributions. First, it provides the first systematic study of the oracle problem in UI repair, establishing baselines for how accurately and how confidently VLM agents verify fixes. Second, it reframes fix verification as grounded inference over a specification (the bug report) rather than perceptual comparison of screenshots. Third, it extends FRAME's component-level UI understanding to a new task: detecting behavioral change guided by natural-language specifications. Fourth, BugFix-Oracle will be the first public benchmark for studying verification reliability on UI bugs. The Sage Research Lab is well positioned for this work: it developed FRAME [1] and already holds the structured bug reports and emulator reproduction scripts from [2].

Broader Impacts

AI coding agents are quickly becoming standard practice, and their adoption assumes they can check their own work; if they cannot, every team relying on automated repair inherits the error. Because AI and cybersecurity are national priority areas, unreliable AI-written code is also a national security and economic competitiveness concern; this project addresses that risk directly. The BugFix-Oracle benchmark and oracle implementation will be released as open-source tools that practitioners can add to CI/CD pipelines. When evaluations expose systematic verification failures in specific AI repair tools, I will report them to the tools' developers so they can be corrected. I will also share the findings through developer-facing talks and plain-language writing.

References

[1] A. K. Vajjala, Y. Yan, A. K. Vajjala, S. Pothagoni, D. Poshyvanyk, and K. Moran, "Fusing UI Structure & Semantics for Feature-Oriented App Screen Retrieval & Clustering," arXiv:2608.08880, 2026.
[2] A. Saha et al., "Automated Program Repair of UI-centric Android Bugs," under review, 2026.